Zero-Rewind Leftmost-Longest Automaton

核心直觉模型、匹配起源三分完备性、持久化发射链结构共享与 $O(|V|)$ 构建规范

daachorse Project Architecture Specification · 2026-10-12

目录

1 引言与问题背景 (Introduction) . . . 1
2 核心直观图像与设计原理 (Core Intuitions & Mental Model) . . . 2
2.1 矛盾根源：为何经典 Aho-Corasick 的 Fail 链不能直接用于最左最长匹配? . . . 2
2.2 核心物理图像：历史沉淀与未来延伸的解耦 . . . 2
2.3 三大核心设计直觉 (Three Intuitive Principles) . . . 2
2.3.1 直觉 1：历史不可篡改（History is Frozen in Amber） . . . 2
2.3.2 直觉 2：局部相对间距恒定与无损结构共享（Local Invariance & DAG Sharing） . . . 3
2.3.3 直觉 3：匹配起源的三分演化（Match Origin Trichotomy） . . . 3
3 形式化符号与基础定义 (Formal Foundations) . . . 3
3.1 最深前缀匹配定义 (`deepest_output`) . . . 3
4 Leftmost-Longest 匹配公理系统 (Axioms) . . . 4
5 匹配起源三分完备性定理 (Match Origin Trichotomy) . . . 4
5.1 一维区间几何划分 . . . 4
5.2 时间展开原理 (Temporal Unrolling Principle) . . . 5
6 子状态继承发射链的核心定理与证明 . . . 5
6.1 定理 6.1：最左优先的历史不可篡改性 . . . 5
6.2 定理 6.2：局部相对位移恒定性 (Local Delta Invariance) . . . 6
6.3 定理 6.3：持久化 DAG 结构共享 (Persistent Structural Sharing) . . . 6
7 递推方程与均摊 $O(1)$ 复杂度证明 (Recurrence Relations) . . . 6
7.1 状态转移二元组与递推方程 . . . 6
7.2 势能均摊分析 (Potential Method) . . . 7
8 运行期流式解码与匹配提取 (Runtime Extraction) . . . 8
9 结论与工程映射 (Summary) . . . 8

1 引言与问题背景 (Introduction)

多模式匹配经典算法（如 Aho-Corasick 算法）主要用于枚举文本中所有匹配的模式（Overlapping Matching）。然而，在中文分词、实体识别（NER）以及正则表达式引擎等实际场景中，普遍采用的是**最左最长匹配语义**（Leftmost-Longest Match Kind）：
1. 优先选取在文本中出现位置最靠左（起始索引最小）的模式；
2. 若多个模式在同一位置起始，优先选取长度最长的模式；
3. 一旦确立匹配 $[s, e)$，匹配指针直接向前跃迁至 e，区间内部所有潜在候选全部被屏蔽。

在传统实现（包括现有工业级 Trie 库）中，当自动机探索到一个局部最深路径后遭遇失配（Mismatch），往往需要**将文本指针向左回溯（Rewind）**，重新从前缀结束点处再次进入自动机重新探测。在精心构造的对抗性文本（Adversarial Input，如重复的重叠候选串）下，扫描时间复杂度会严重退化至 $O(L \cdot N)$（L 为最长词长度，N 为文本长度）。

本规范形式化定义了 **Zero-Rewind Leftmost-Longest Automaton**，通过在 Trie 树构建期预计算**持久化发射链（Persistent Emit Chains）**与**精化失败转移（Refined Fail Transitions）**，将文本匹配的扫描过程严格约束在 $O(N)$ 时间复杂度内（零文本回溯、零运行期堆内存分配）。同时，本规范从数学上严格论证：

- 为什么模式树中任意节点的匹配起源严格满足三**分完备性定理（Match Origin Trichotomy）**；
- 为什么子状态可以安全且无损地**结构共享（Structural Sharing）**祖先节点的发射链；
- 为什么整个自动机的构建期均摊耗时严格收敛至 $O(|V|)$。

—

2 核心直观图像与设计原理 (Core Intuitions & Mental Model)

在展开深入的数学符号与定理证明之前，本节首先阐述驱动算法设计的物理图像与直观原理，使整篇规范完全自包含（Self-contained）。

2.1 矛盾根源：为何经典 Aho-Corasick 的 Fail 链不能直接用于最左最长匹配?

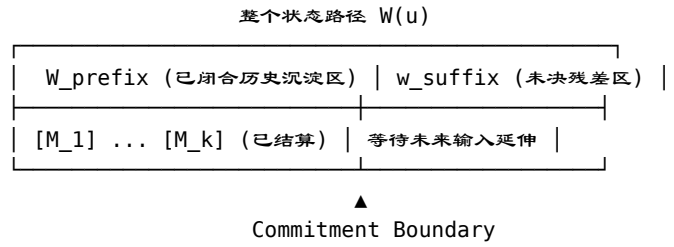
在经典 Aho-Corasick（AC）算法中，当失配发生时，自动机会沿着失败链接（Fail Link）跳向当前路径的“最长真后缀”（Longest Proper Suffix）。这在查找“全部子串”时是完全正确的。

但在 **Leftmost-Longest** 语义下，这种机制会直接失效：

- **最左优先具有“一票否决权”**：如果在较早位置已经确认了一个匹配模式 M_1，那么 M_1 区间内部所包含的一切字符全部被屏蔽作废；
- 如果直接沿着真后缀跳转，自动机会盲目拾取那些本应被更靠左的长词彻底杀死的“影子子串”，违背最左贪心原则；
- 传统工程实现为了不采错模式，只能在失配时将文本指针暴力回退（Rewind）到上一个合法词的末尾重新开始。这种回溯在对抗性文本下造成了严重的性能悬崖（退化为 $O(L \cdot N)$）。

2.2 核心物理图像：历史沉淀与未来延伸的解耦

零回溯算法的核心直觉，是在自动机沿状态路径 $W(u)$ 行进时，从物理上将路径划分为两个互斥的区域：



1. **已闭合历史沉淀区 (W_{prefix})**：位于结算边界左侧。这里的模式序列 $[M_1, ..., M_k]$ 已经彻底结算。因为从它们各自的终点出发的分支此前已经在 Trie 中断裂失配，未来的字符绝对无法再回头延伸它们。
2. **未决残差区 (w_{suffix})**：位于结算边界右侧。这是一截尚在 Trie 中拥有可用出边的活跃真后缀。自动机正在代表这截后缀贪心地试探未来输入，期待它能在后续字符中被拼成一个更长的模式。

2.3 三大核心设计直觉 (Three Intuitive Principles)

算法的一切数学性质与数据结构设计，皆源自以下三大直观原理：

2.3.1 直觉 1：历史不可篡改（History is Frozen in Amber）

- **直觉疑问**：子状态多吃进一个新字符 c 变成 $W(u) \cdot c$，会不会逆向推翻父状态原来认定的旧匹配?
- **直观解答**：绝对不可能！ 任何依赖新字符 c 的新模式，终点必然在最右端。它的起点要么在历史匹配之后（不影响历史），要么在历史匹配内部（被历史匹配的最左优先一票否决），要么在历史匹配之前（若能在之

前起步且延伸到 c，说明该分支此前从未失配，算法在过去根本不会提前结算旧匹配）。 因此：**过去闭合的匹配像冻结在琥珀中一样，拥有绝对的单向不可逆性**。

2.3.2 直觉 2：局部相对间距恒定与无损结构共享（Local Invariance & DAG Sharing）

- **直觉疑问**：子状态比父状态深度更深，绝对坐标全变了，为什么子状态能直接用指针指向父状态的发射链？
- **直观解答**：因为链表中存储的不是全局绝对坐标，而是**相邻两个匹配之间的局部距离（Gap）**！ 两个已经闭合的历史匹配 M_1 与 M_2 之间的距离，纯粹是它们中间夹着的那段未匹配字符的长度。无论后来又追加了多少个字符、无论树分叉出多少万条路径，这个间距是永恒不变的常数。 因此，历史发射链就像函数式编程中的持久化单向链表（Persistent Cons List）一样，子状态只需要分配 1 个新头节点（甚至 0 个），剩下的整条历史链在所有子孙分叉之间 100% 零拷贝共享。

2.3.3 直觉 3：匹配起源的三分演化（Match Origin Trichotomy）

- **直觉疑问**：为什么我们在构建时完全不需要像传统做法那样反向扫描整个字符串去寻找匹配？所有可能被采纳的匹配到底从何而来？
- **直观解答**：任何一个在 $W(u)$ 中成立的匹配，在它闭合的那个字符处，当时它必然处于字符串的最末端（要么是整串的前缀，要么是当前的临界后缀）。随着树枝向下生长，它被自然封存在链表里，作为历史传给后代。 因此，所有匹配严格来自三类：**根前缀、临界后缀**、或**祖先历史继承**。世界上根本不存在凭空在内部突然冒出来的第四类匹配。

—

3 形式化符号与基础定义 (Formal Foundations)

符号	数学类型	物理含义	代码对应
Σ	Alphabet	字符集（UTF-8 字符或单字节）	`char` / `u8`
$\mathcal{P}$	Pattern Set	模式集 $\mathcal{P} = \{p_1, \ldots, p_m\} \subset \Sigma^+$	输入模式列表
$T = (V, E, \text{ROOT})$	Trie	模式集 $\mathcal{P}$ 构造的前缀树	`NfaBuilder`
$u, v \in V$	States	Trie 树中的状态节点	`state_id: u32`
ROOT	Root State	根状态，深度为 0，代表空字符串 ε	`ROOT_STATE_ID` (0)
$W(u) \in \Sigma^*$	Path String	从 ROOT 走到节点 u 的唯一路径字符串	节点对应的前缀
$d(u) = \lvert W(u) \rvert$	Depth	节点 u 的深度（字符串长度）	`depth[u]`
$\delta(u, c)$	Transition	状态转移：沿字符 c 的出边，无出边则为 $\perp$	`child_id(u, c)`
$\text{edges}(u)$	Edge Set	状态 u 的所有有效出边集合	`states[u].edges`
$\text{Output}(u)$	Output	若 $W(u) \in \mathcal{P}$，则表示模式匹配终点	`states[u].output`

3.1 最深前缀匹配定义（`deepest_output`）

> **定义 3.1 (最深前缀匹配 deepest_output)**
>
> 对于树中任意状态 $u \in V$（对应路径字符串 $W(u)$），从 ROOT 到 u 的路径上包含的所有祖先节点中，**深度最大（对应模式串最长）且具有模式终点标记**的节点，记作 $\text{deepest_output}(u)$：

> $$\text{deepest_output}(u) = \arg \max_{v \in \text{ Ancestors}(u) \cup \{u\},\ \text{Output}(v) \neq \varnothing} d(v)$$
>
> 若整条路径上无任何合法模式，则 $\text{deepest_output}(u) = \perp$。

物理语义：$\text{deepest_output}(u)$ 唯一回答了以下核心问题：“从位置 *0* 开始匹配，沿着路径 $W(u)$ 走，所能构成的最长合法模式是哪一个？”

构建期 $O(1)$ 递推计算：在广度优先遍历（BFS）队列推进时，每个子节点的 deepest_output 仅取决于自身是否为模式终点以及父节点的最深匹配：

$$\text{deepest_output}(v) = \begin{cases} v & \text{若 } \text{Output}(v) \neq \varnothing \\ \text{deepest_output}(u) & \text{若 } \text{Output}(v) = \varnothing \end{cases}$$

每个节点仅需一次条件判断，全树预计算耗时严格为 $O(|V|)$。

—

4 Leftmost-Longest 匹配公理系统 (Axioms)

1. **最左优先公理 (Leftmost Priority)**：若候选匹配 $M_A = [s_A, e_A)$ 与 $M_B = [s_B, e_B)$ 存在冲突，且 $s_A < s_B$，则 M_A 具有绝对仲裁权，M_B 必须被舍弃。
2. **最长优先公理 (Longest Priority)**：若 $s_A = s_B$ 且 $e_A > e_B$，则 M_A 胜出，M_B 被舍弃。
3. **贪心跃迁与区间屏蔽 (Greedy Shielding)**：一旦确立匹配 $M = [s, e)$，区间 $[s, e)$ 内部起始的所有后续字符全部被屏蔽，匹配指针直接跃迁至 e。

—

5 匹配起源三分完备性定理 (Match Origin Trichotomy)

在分析状态路径字符串 $W(u)$（长度为 L）所包含的所有合法匹配时，本节严格证明其匹配起源在数学上满足完备的三分划分（Trichotomy）。

5.1 一维区间几何划分

对于给定的任意路径字符串 $W(u)$，任何被采纳的匹配 $M = [s, e)$ 均为整数区间 $[0, L]$ 上的子区间，满足 $0 \leq s < e \leq L$。

考察区间端点与字符串两端边界的接触状态：

- 起点 s：要么 $s = 0$（接触左端点），要么 $s > 0$（脱离左端点）；
- 终点 e：要么 $e = L$（接触右端点），要么 $e < L$（脱离右端点）。

对这两组二元状态作笛卡尔积，在拓扑上严格穷尽以下四种几何形态：

几何条件	边界接触	物理类型	收敛归类
$s = 0, e = L$	双端接触	全串覆盖匹配	类别 1 (根锚定前缀)
$s = 0, e < L$	仅贴左端	根前缀真子串匹配	类别 1 (根锚定前缀)
$s > 0, e = L$	仅贴右端	当前临界真后缀匹配	类别 2 (临界真后缀)
$s > 0, e < L$	双端脱离	严格内部匹配	类别 3 (历史内部继承)

$(s = 0) \cup (s > 0 \wedge e = L) \cup (s > 0 \wedge e < L) \equiv \{[s, e) \subseteq [0, L] \mid 0 \leq s < e \leq L\}$。 几何上，任意子区间严格落入上述三类之一，分类无任何盲区与死角。

5.2 时间展开原理 (Temporal Unrolling Principle)

定理 5.1 (匹配起源三分完备性定理)

在状态树的任意节点 u 处，任何一个被 Leftmost-Longest 语义采纳的匹配 $M = [s, e)$，其被自动机识别并定格的时机满足且仅满足以下三者之一：

1. **类别 1：根锚定匹配 ($s = 0$)**：M 是路径的前缀，由 deepest_output(u) 在 $O(1)$ 时间内唯一定位。
2. **类别 2：临界后缀匹配 ($s > 0, e = |W(u)|$)**：M 是当前路径的真后缀，由经典 Aho-Corasick 后缀链接链（ac_fail）上的输出节点在 $O(1)$ 时间内唯一定位。
3. **类别 3：历史内部匹配 ($s > 0, e < |W(u)|$)**：M 已经在深度为 e 的祖先节点处完成定格，并作为持久化链表节点由父状态直接继承。

证明 (时间展开视角)：对于类别 3 中的任意内部匹配 $M = [s, e)$，其终点满足 $e < |W(u)|$。根据 Trie 树的拓扑性质，状态 u 的路径前缀 $W(u)[0..e]$ 必定对应于树中的唯一祖先节点 u_e（即 $d(u_e) = e$）。在 BFS 遍历或者字符逐步推进的时间线上，系统必定曾经在历史时刻访问过状态 u_e。在访问 u_e 的那一瞬间，匹配 M 的终点恰好等于当时的当前深度（$e = d(u_e)$）。因此，在历史状态 u_e 处，M 必定曾经作为**类别 1（若 $s = 0$）**或**类别 2（若 $s > 0$）**被捕获并结算。随着深度由 e 继续增加至 $|W(u)|$，M 被封存入 u_e 的发射链中，并沿着树枝不可变地传递给所有后代。 □

> 核心推论：类别 3（历史内部匹配）并不是某种独立的生成机制，它不过是祖先节点在它自己的深度时所经历的类别 1 或类别 2 在时间轴上的沉淀。全树构建过程中，不存在任何匹配需要通过“反向逐字扫描历史字符串”来被发现。

—

6 子状态继承发射链的核心定理与证明

子状态 v 为父状态 u 沿字符 c 的后继（$W(v) = W(u) \cdot c$）。本节严格证明子状态直接指向父状态的发射链在数学上的自洽性。

6.1 定理 6.1：最左优先的历史不可篡改性

定理 6.1 (已闭合匹配不可篡改定理)

设在父状态 u 处已经结算并闭合了匹配序列 $\mathcal{M}(u) = [M_1, ..., M_k]$，其中 $e(M_k) \le |W(u)|$。当沿字符 c 进入子状态 $v = \delta(u, c)$ 时，追加字符 c 绝对不可能导致 $\mathcal{M}(u)$ 中的任何匹配被撤销、截断或改变。

反证法证明：假设追加字符 c 后，在子状态 v 的完整路径 $W(v)$ 中出现了一个新的模式匹配 $M^* = [s^*, e^*)$，能够推翻或替代已闭合匹配 $M_i = [s_i, e_i) \in \mathcal{M}(u)$。

根据 Leftmost-Longest 匹配公理，M^* 要想取代 M_i，必须满足以下两项仲裁条件之一：
- **条件 A（最左优先抢占）**：$s^* < s_i$（M^* 起始位置更早）；
- **条件 B（最长优先抢占）**：$s^* = s_i$ 且 $e^* > e_i$（起始位置相同，但 M^* 跨度更长）。

由于 M^* 在父状态 $W(u)$ 中未曾胜出，它必然依赖于最新追加的字符 c，即 $e^* = |W(v)|$。现在检验条件 A 与条件 B 的相容性：

1. **检验条件 B ($s^* = s_i$)**：若存在模式 M^* 始于 s_i 且终于 $|W(v)|$，说明在 Trie 树中，从位置 s_i 开始的分支能够连续匹配文本并延伸至字符 c。若如此，在处理父状态 u 时，始于 s_i 的探测分支尚未失配。按照贪心最长匹配语义，自动机在父状态处处于未决的探索态，**绝不会提前将 M_i 闭合固化**。这与前提“M_i 已经在父状态处闭合结算”发生直接矛盾。

2. **检验条件 A ($s^* < s_i$)**：若存在模式 M^* 始于更早的位置 $s^* < s_i$ 并一直延伸至 c。同理，当扫描经过 s^* 时，自动机必将持续追踪该分支。如果该分支是有效模式，它将在 s_i 之前保持活跃，根本不可能在位置 s_i 允许 M_i 成立；若该分支在此前已失配断裂，则它不可能在末尾包含字符 c 处重新构成完整模式。矛盾。
3. **考察 $s^* > s_i$ 的情形**：若 $s^* \in [s_i, e_i)$，根据最左屏蔽公理，由于 $s_i < s^*$，M_i 拥有一票否决权，M^* 被直接屏蔽废弃；若 $s^* \geq e_i$，M^* 严格处于 M_i 的右侧，二者互不重叠，此时 M^* 仅能作为后续的新增匹配，绝不影响 M_i 本身的有效性。

综上所述，假设不成立。已闭合的历史匹配序列具有绝对的单向不变性。 □

6.2 定理 6.2：局部相对位移恒定性 (Local Delta Invariance)

定理 6.2 (内部 Gap 代数恒定性)

对于发射链中的任意两个相邻历史匹配 $M_i = [s_i, e_i)$ 与 $M_{i+1} = [s_{i+1}, e_{i+1})$，定义其内部相对间距为：

$$\text{gap}(M_{i+1}) = s_{i+1} - e_i$$

则 $\text{gap}(M_{i+1})$ 仅取决于区间 $[e_i, s_{i+1})$ 内未匹配文本的字节长度，与后续追加的字符总数、任意子树分叉以及当前状态的绝对深度 $d(v)$ 完全无关。

证明：e_i 与 s_{i+1} 均为路径中固定字符位置的绝对偏移。由定义，$\text{gap}(M_{i+1}) = s_{i+1} - e_i$ 是两个已闭合区间之间的固定常数。当状态从 u 演化到后代状态 v 时，新增字符全部追加在 $s_{i+1} < e_{i+1} \leq |W(u)| < |W(v)|$ 之后，无法改变前缀区间坐标。因此该相对位移是整棵子树拓扑共享的绝对不变量。 □

6.3 定理 6.3：持久化 DAG 结构共享 (Persistent Structural Sharing)

基于定理 6.1 与定理 6.2，发射链采用逆向单向链表表示：

```
Head (最新匹配 M_k) —(gap)—> Predecessor (M_{k-1}) —(gap)—> ... —> M_1 —> None
```

每个链表节点定义为四元组：

$$\text{Node} = \langle \text{value}, \text{length}, \text{gap}, \text{pred} \rangle$$

当父状态 u 分叉出多个子节点 v_1, v_2, v_3 时：

- 若子分支未产生新闭合匹配（残差继续延伸）：

$$\text{output_pos}(v) = \text{output_pos}(u)$$

  直接共享父节点指针，耗时 $O(1)$，内存增量为 0。
- 若子分支闭合了新的匹配 M_{new}：分配单一新节点 N_{new}，设置其前驱指针：

$$N_{\text{new}}.\text{pred} = \text{output_pos}(u)$$

  将其作为新头部存入 $\text{output_pos}(v)$。原前驱链 $M_k \to \ldots \to M_1$ 在所有同源分支之间完全复用。

整个状态空间的发射链构成了一个全局只读的有向无环图（Directed Acyclic Graph, DAG），消除了所有多余的节点拷贝。

—

7 递推方程与均摊 $O(1)$ 复杂度证明 (Recurrence Relations)

7.1 状态转移二元组与递推方程

每个状态的核心元组表示为：

$$\mathcal{S}(u) = (S_{\text{suffix}}(u), \mathcal{E}(u))$$

其中 $S_{\text{suffix}}(u) \in V$ 为未决的残差后缀状态，$\mathcal{E}(u)$ 为持久化发射链头指针。

对于子节点 $v = \delta(u, c)$，令父节点后缀状态为 $s = S_{\text{suffix}}(u)$：

1. **分支 1 (全局根匹配覆盖 Fast-Path)**：若 $p_0 = \text{deepest_output}(v) \neq \bot$ 且 $d(p_0) = d(v)$：
$$S_{\text{suffix}}(v) = \text{ROOT}, \quad \mathcal{E}(v) = \text{Cons}(p_0, \varnothing) \quad [O(1)]$$
2. **分支 2 (残差后缀自然延伸 Fast-Path)**：令 $s' = \delta(s, c)$。若 $s' \neq \bot$ 且 $\text{edges}(s') \neq \varnothing$：
$$S_{\text{suffix}}(v) = s', \quad \mathcal{E}(v) = \mathcal{E}(u) \quad [O(1)]$$
3. **分支 3 (残差断裂与回退 Slow-Path)**：若 s' 无法继续延伸，则沿已构建的后缀链回溯 $\text{curr} = s$：
   - 若 $\delta(\text{curr}, c)$ 是模式终点 M：
   $$\mathcal{E}(v) = \text{Cons}(M, \mathcal{E}(u)), \quad S_{\text{suffix}}(v) = \text{ROOT}$$
   - 若 $\delta(\text{curr}, c)$ 有可用出边：
   $$S_{\text{suffix}}(v) = \delta(\text{curr}, c), \quad \mathcal{E}(v) = \mathcal{E}(u)$$
   - 否则回跳 $\text{curr} \leftarrow S_{\text{suffix}}(\text{curr})$ 并重复检查。

7.2 势能均摊分析 (Potential Method)

> **定理 7.1 (线性构建复杂度定理)**
>
> 全树状态在广度优先遍历构建过程中，分支 3 执行后缀回跳的总步数严格受限于状态总数 $|V|$。每个状态的构建均摊时间复杂度为 $O(1)$，整棵树的构建总复杂度严格为 $O(|V|)$。

证明：定义全树状态的势能函数为残差后缀状态的深度：

$$\Phi(u) = d(S_{\text{suffix}}(u))$$

显然，对于根状态 $\Phi(\text{ROOT}) = 0$，且对任意状态 $\Phi(u) \geq 0$。

考察树枝推进 $u \to v = \delta(u, c)$ 引起的势能变化：

1. 在分支 2（后缀延伸成功）中，深度最多增加 1：
$$\Phi(v) \leq \Phi(u) + 1$$
遍历全树 $|V| - 1$ 条边，势能增加的总上限严格满足：
$$\sum_{v \in V} \Delta\Phi^+ \leq |V|$$
2. 在分支 3（失配回退）中，每次执行回跳操作 $\text{curr} \leftarrow S_{\text{suffix}}(\text{curr})$：由于真后缀的深度严格小于原状态深度：
$$d(S_{\text{suffix}}(\text{curr})) \leq d(\text{curr}) - 1$$
每回跳一步，势能至少减少 1。
3. 鉴于势能非负（$\Phi \geq 0$），在整个算法生命周期内，势能减少的总量绝对不能超过势能增加的总总量：
$$\sum_{v \in V} (\text{回跳步数}) \leq \sum_{v \in V} \Delta\Phi^+ \leq |V|$$

因此，回跳的总步数在全树均摊下为 $O(1)$。整棵自动机的构建时间严格为 $O(|V|)$。 □

—

8 运行期流式解码与匹配提取 (Runtime Extraction)

在运行期，自动机扫描文本 T，当前扫描指针为 P。当在某个状态命中需要输出的发射链 $\mathcal{E}$ 时，执行无堆分配的逆向坐标复原：

```
// 逆向差分复原算法（严格 O(1) 每匹配，零堆内存分配）
let head = &emits[output_pos - 1];
let mut end = pos - head.gap;
let mut start = end - head.length;
buf.push(head.to_match(start, end));

let mut curr = head.pred;
while let Some(pred_idx) = curr {
    let em = &emits[pred_idx - 1];
    end = start - em.gap;       // 利用上一匹配的 start 减去相对 gap
    start = end - em.length;
    buf.push(em.to_match(start, end));
    curr = em.pred;
}
// 从 LIFO 缓冲栈中反向弹出，流式输出正序匹配序列 [M_1, M_2, ..., M_k]
```

- **空间开销**：复用线程局部的栈缓冲区 `buf: Vec<Match>`，无任何运行时堆分配；
- **时间开销**：每个匹配项解算仅耗费 2 次减法与 1 次数组索引，时间严格为 $O(k)$；
- **文本扫描**：指针 pos 单向递增，绝不执行后退（Zero Rewind），文本扫描总耗时严格为 $O(N)$。

—

9 结论与工程映射 (Summary)

本文档确立了 **Zero-Rewind Leftmost-Longest Automaton** 的完备数学基础：

1. **完备性**：匹配起源三分完备性定理证明了不存在脱离“根前缀”、“临界后缀”与“历史继承”之外的第四类匹配，否定了暴力回溯的必要性；
2. **正确性**：最左优先的历史不可篡改性与局部 gap 恒定性保证了子状态可以安全指向祖先节点的发射链；
3. **高效性**：势能分析法证明了 $O(|V|)$ 的构建复杂度，持久化 DAG 保证了紧凑的内存开销，差分逆向链实现了严格 $O(N)$ 的零回溯匹配。